Practical: 1	Implementation and Time analysis of sorting algorithms. Bubble sort, Selection sort, Insertion sort, Merge sort and Quicksort

Aim: To Implement and Time analysis of sorting algorithms. Bubble sort, Selection sort, Insertion sort, Merge sort and Quicksort.

Algorithm:

i) Bubble sort Algorithm

Input : unsorted Array
output : sorted Array
step 1 : for i=0 in condition i<n then do
step 2 : for j=0 in condition of j<n-i-1 the
do
step 3 : if arr[j] > arr[j+1] then [swap]
swapping
temp = arr[j]
arr[j] = arr[j+1]
arr[j+1] = temp
step 4 : increment the inner loop of i++ iterat
step 5 : increment the outer loop of i++ iterat on
step 6 : stop

(ii) Selection sort:

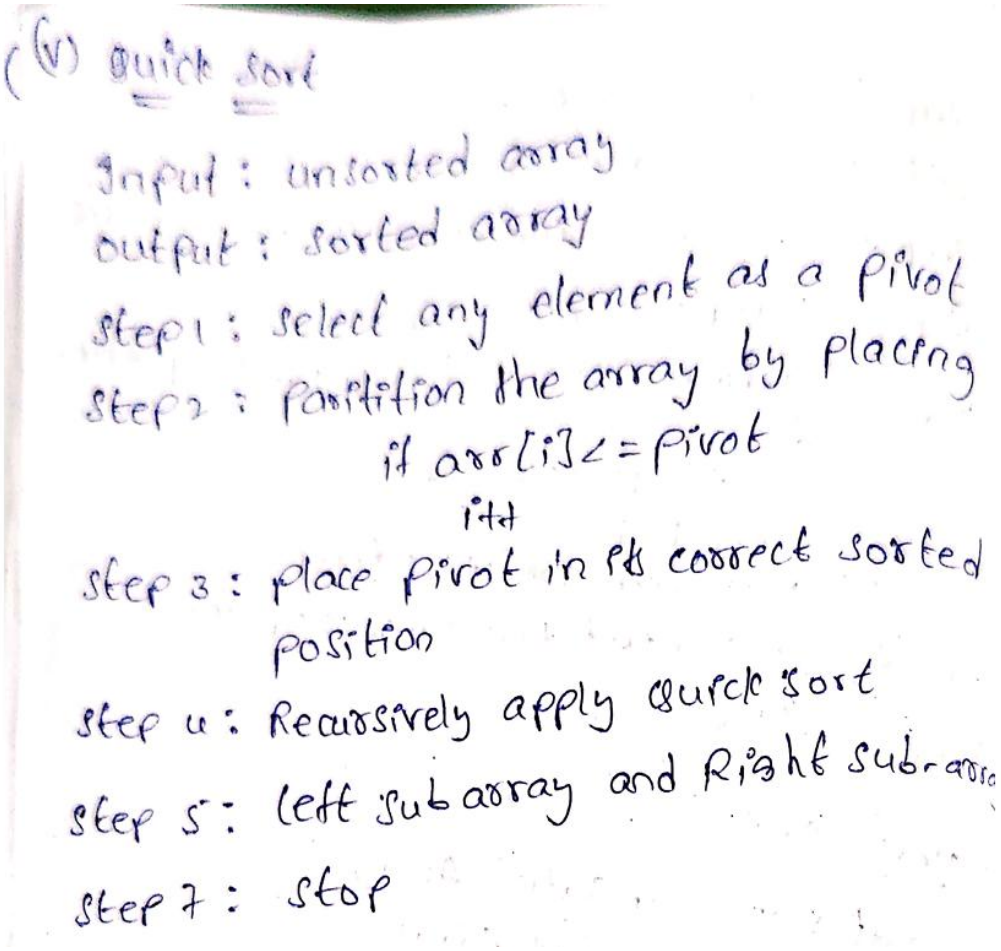
Input: unsorted Array
output : sorted Array
step 1 : for i=0 in the condition i<n then do
step 2 : min=i and inner for loop initialize j= i+1
step 3 : in the condition j<n then do if arr[j] > arr
[min] then min=j
step 4 : increment the inner loop j++
step 5 : if (min!=0 then
swap(arr[i], arr[min])
step 6 : increment the outer loop i++
step 7 : stop

(ii) Insertion sort:

Input : unsorted array
output : sorted array
step 1 : for i=1 in the condition i<n then do
step 2 : temp = arr[i] and j = i-1
step 3 : while j>=0 and arr[j]>temp then do
arr[j+1] = arr[j]
j = j-1
step 4 : initilaze arr[j+1] = temp
step 5 : increment the outer loop i++
step 6 : stop

(iv) Merge sort:

Input sort : unsorted Array
output : sorted Array
step 1 : check if array has more than one element
step 2 : Divide the array into two equal halves.
step 3 : Recursively sort the left half using merge sort
step 4 : same for Right half using merge sort
step 5 : merge the two sorted halves into a single sorted array
step 6 : stop.



Program:

1. Bubble sort, selection sort, Insertion sort

```
#include<iostream>
using namespace std;

class classname{
        public:
                void Func(){
                int array1[100];
                int input,i;
                cout<<"Enter the size of array: ";
                cin>>input;
                array1[input];
                cout<<"Enter the Elements in array:\n";
                for(int i=0; i< input; i++){
                        cin>>array1[i];
                }
                int n = sizeof(array1)/sizeof(array1[0]);
                display(array1,input);
                cout<<"\n\n";
                Bubble(array1,input);
```

```
		cout<<"\n\n";
		selection(array1,input);
		cout<<"\n\n";
		insertion(array1,input);
		cout<<"\n\n";
	//	quickSort(array1, 0, n-1);
	}
	void display(int array1[],int n){
		cout<<"Your Array is:\n";
		for(int i = 0; i<n; i++){
			cout<<array1[i]<<" ";
		}

	}
	void Bubble(int array1[],int n){
		cout<<"Bubble sorted array:\n";
		for(int i=0;i<n;i++){
			for(int j=0; j<n-i-1; j++){
				if(array1[j] > array1[j+1]){
					int temp = array1[j];
					array1[j] = array1[j+1];
					array1[j+1] = temp;
				}

			}
		}
		display(array1,n);
	}
	void selection(int array1[],int n){
		cout<<"selection sort Array:\n";
		for(int i = 0; i<n; i++){
			int min = i;
			for(int j=i+1; j<n; j++){
				if(array1[j] < array1[min]){
					min = j;
			}
			if(min != 0){
				int temp = array1[i];
				array1[i] = array1[min];
				array1[min] = temp;
			}
		}
	}
	display(array1,n);
}

	void insertion(int array1[],int n){
	cout<<"insertion sort Array:\n";
```

```
        for(int i=1;i<n;i++){
                int temp = array1[i];
                int j = i-1;
                while(j>=0 && array1[j]>temp){
                        array1[j+1] = array1[j];
                        j=j-1;
                }
                array1[j+1] = temp;

        }
        display(array1,n);
}

int main(){
        classname obj;
        obj.Func();
        return 0;
}
```

2. Quick sort and merge sort:

```
#include<iostream>
using namespace std;

//------------------- Swap Function -------------------
void swap(int* a, int* b) {
   int temp = *a;
   *a = *b;
   *b = temp;
}

//------------------- Quick Sort -------------------
int partition(int arr[], int start, int end) {
   int pivot = arr[end];
   int i = start - 1;

   for (int j = start; j < end; j++) {
      if (arr[j] <= pivot) {
         i++;
         swap(&arr[i], &arr[j]);
      }
   }

   swap(&arr[i + 1], &arr[end]);
   return i + 1;
}

void quickSort(int arr[], int start, int end) {
   if (start < end) {
```

```
        int pi = partition(arr, start, end);

        quickSort(arr, start, pi - 1);
        quickSort(arr, pi + 1, end);
    }
}

//------------------- Merge Sort -------------------
void merge(int arr[], int left, int mid, int right) {
    int n1 = mid - left + 1;
    int n2 = right - mid;

    int L[n1], R[n2];

    // Copy data to temporary arrays
    for (int i = 0; i < n1; i++)
        L[i] = arr[left + i];

    for (int j = 0; j < n2; j++)
        R[j] = arr[mid + 1 + j];

    int i = 0, j = 0, k = left;

    // Merge the temporary arrays
    while (i < n1 && j < n2) {
        if (L[i] <= R[j]) {
            arr[k] = L[i];
            i++;
        }
        else {
            arr[k] = R[j];
            j++;
        }
        k++;
    }
    // Copy remaining elements
    while (i < n1) {
        arr[k] = L[i];
        i++;
        k++;
    }

    while (j < n2) {
        arr[k] = R[j];
        j++;
        k++;
    }
}
```

```
void mergeSort(int arr[], int left, int right) {
    if (left < right) {
        int mid = (left + right) / 2;
        mergeSort(arr, left, mid);
        mergeSort(arr, mid + 1, right);
        merge(arr, left, mid, right);
    }
}
void display(int array1[], int n) {
    for (int i = 0; i < n; i++) {
        cout << array1[i] << " ";
    }
    cout << endl;
}
int main() {
    int arr[] = {9, 9, 7, 78, 5};
    int n = sizeof(arr) / sizeof(arr[0]);
    cout << "Original Array:\n";
    display(arr, n);
    //Quick Sort
    quickSort(arr, 0, n - 1);
          cout<<endl;
    cout<<"Quicksort sorted array: ";
    display(arr, n);
    // Merge Sort
    mergeSort(arr, 0, n - 1);
    cout<<endl;
    cout<<"mergesort sorted array: ";
    display(arr, n);
    return 0;
}
```

Output:

```
Enter the size of array: 5
Enter the Elements in array:
1 9 87 4 5
Your Array is:
1 9 87 4 5

Bubble sorted array:
Your Array is:
1 4 5 9 87

selection sort Array:
Your Array is:
1 4 5 9 87

insertion sort Array:
Your Array is:
1 4 5 9 87
```

2.

```
Original Array:
9 9 7 78 5

Quicksort sorted array: 5 7 9 9 78

mergesort sorted array: 5 7 9 9 78
```

Time Complexity:

Time complexity :-

```
for(i=0; i<n-1; i++) {          → n-1
for(j=0; j<n-1; j++) {          → n-i-1
  if (arr[j] > arr[j+1] {
    swap (arr[j] arr[j+1]);
  }
}
}
```

$T(n) = (n-1) + (n-2) + \dots + 1 \Rightarrow (n-1) + (n-2) + \dots + 1$

$= \frac{n(n-1)}{2} \Rightarrow O(n^2)$

$= O(n^2)$

Time complexity (selection sort)

```
for(int i=0 ; i<n-1; i++) {     → n
   int minIndex = i;            → 1
for(int j=0; j<n; j++) {        → n-i-1
    if (arr[j] < arr[minIndex]) {
        minIndex = j;
    }
}
int temp=arr[i];                → 1
arr[i]= arr[minIndex];          → 1
arr[minIndex]= temp;            → 1
```

Total :

$n + n-i-1$

$(n-1) + (n-2) + \dots + 1$

$n(n-1)/2 = O(n^2)$

$\Rightarrow O(n^2)$

Time complexity (Insertion sort)

```
for (int i=1; i<n; i++) {          ---> n
    int key = arr[i];              ---> 1
while (j>=0 && arr[j]>key){        ---> i-1
    arr[j+1] = arr[j];             ---> 1
        j--; }
arr[j+1] = key; }
```

Total:

$1+2+3+\ldots\ldots+(n-1)$

$\Rightarrow n(n-1)/2$

$= O(n^2)$

Time complexity (Merge sort)

```
void mergesort (int a[], int l, int r) {
    if (l<r) {
    int m=(l+r)/2;
merge sort (a,l,m); merge sort(a,m,l,r);
    merge(a,l,m,r); }
    }
```

Merge sort recurrence:..

$T(n) = 2T(n/2) + O(n)$

compare with

$T(n) = aT(n/b) + f(n)$

$a=2 \quad k=1$

$b=2 \quad f(n)=n$

$p=0$

$\log_2^2 = 1$

$k = \log_2^2$

So, here $f(n) = \log_b^a$

So,

$T(n) = (n \log n)$

$T(n) = O(n \log n)$

continue until the ...

```
void quick sort (int a[], int low, int high)
    if (low<high) {
      int p= partition (a, low, high);
    quick sort (a, low, p-1);
    quick sort (a, p+1, high);
    }
}
```

partition o(n)

quick sort (left) + quick sort (right)

$T(n) = 2T(n/2) + o(n)$

By using master method

$a=2, b=2, f(n)=n, k=1, p=0$

$\log_2^2 = 1$

By comparing k & $\log_b^a$ value $k = \log_b^a$

so, $T(n) = n \log n$

$\boxed{T(n) = n \log n}$ → Best / Average case

$\boxed{T(n) = o(n)^2}$ → worst case.

Conclusion: Hence, By Implementing and analysing of sorting algorithms. Bubble sort, Selection sort, Insertion sort, Merge sort and Quicksort.
The time complexity plays an important role in the logical part of the program. And the Programs are successfully executed.